Python Foundations Cheat Sheet

Lesson 1 — Variables, Data Types, Operators & Control Flow

Core Data Types

Type	Example	Mutable?	Notes
int	x = 5	N/A (immutable)	Whole numbers, arbitrary precision
float	x = 5.0	N/A (immutable)	Decimal numbers; watch for rounding errors
str	x = "hi"	No	Immutable sequence of characters
bool	x = True	N/A (immutable)	True/False; subtype of int (True == 1)
list	x = [1, 2, 3]	Yes	Ordered, indexable, changeable
tuple	x = (1, 2, 3)	No	Ordered, indexable, fixed once created
dict	x = {"a": 1}	Yes	Key-value pairs, insertion-ordered (3.7+)
set	x = {1, 2, 3}	Yes	Unordered, no duplicates

Operators

Arithmetic: + - * / // (floor div) % (modulo) ** (power) **Comparison:** == != < > <= >= **Logical:** and / or / not **Assignment shortcuts:** += -= *= /=

/* vs *//: `7 / 2` = 3.5 (true division, always float) `7 // 2` = 3 (floor division, drops the remainder)

Control Flow

if / elif / else — for <item> in <iterable> — while <condition> — break (exit loop now) — continue (skip to next iteration)

```
for i, item in enumerate(my_list):
    print(i, item)

x = 5 if condition else 10  # ternary expression
```

Truthy / Falsy Values

Falsy: `0, 0.0, "", [], {}, (), set(), None, False` **Everything else is truthy** — including the string `"0"` and an empty-looking-but-non-empty object.

Common Pitfalls (these will trip you up — know them now)

- Strings and tuples are **immutable** — `my_str[0] = 'x'` raises a TypeError. Build a new one instead.
- Lists are **mutable** — assigning `b = a` (both lists) makes b point to the *same* list, not a copy. Use `b = a.copy()` or `b = list(a)` for a real copy.
- == checks value equality; `is` checks identity (same object in memory). Use == unless you specifically need identity (e.g. `x is None`).
- Integer division in Python 3 with a single `/` always returns a float, even `4 / 2` = 2.0, not 2.
- Dictionaries preserve insertion order (Python 3.7+), but don't rely on this for anything requiring true sorting — use `sorted()` explicitly.

Bootcamp Lesson 1 of N — Software Engineering → AI Solutions Architect / FDE track. See lessons/lesson-01-python-foundations/README.md for the full Watch/Read/Reinforce/Build kit.